Educational-Platform Documentation

Introduction

This project implements a gamified and personalized educational platform with a database that adapts content and interactions based on learners' personality, emotions, health, and background. It integrates gamification (badges, quests, leaderboards) to boost engagement while ensuring scientifically grounded personalization. The database manages learner data, course content, interactions, gamification, and analytics.

For setup instruction, refer to the README in the GitHub repository

Navigation

There are 2 main directories for in the project

1. SRC Queries

Contains all the T-SQL queries

1. TableScript.sql

This script contains the SQL queries to create the tables necessary for the educational platform. Below is a list of all tables and their purposes:

- Learner: Stores information about learners.
- Skills: Stores skills of learners.
- LearningPrefrences: Stores learning preferences of learners.
- PersonalProfile: Stores personal profiles of learners.
- HealthCondition: Stores health conditions of learners.
- Course: Stores information about courses.
- CoursePrerequisites: Stores prerequisites for courses.
- Module: Stores information about modules within courses.
- TargetTraits: Stores target traits for modules.
- ModuleContent: Stores content types for modules.
- ContentLibrary: Stores library content related to modules.
- Course_Enrollment: Stores enrollment information for learners in courses.
- Assessment: Stores assessment information for modules.

- TakenAssessment: Stores information about assessments taken by learners.
- Instructor: Stores information about instructors.
- LearningPath: Stores learning paths for learners.
- pathreview: Stores reviews of learning paths by instructors.
- learningActivity: Stores learning activities within modules.
- Emotional_feedback: Stores emotional feedback from learners.
- Emotionalfeedback_review: Stores reviews of emotional feedback by instructors.
- Teaches: Stores information about which instructors teach which courses.
- Notification: Stores notifications for learners.
- RecivedNotfy: Stores information about received notifications.
- Reward: Stores reward information.
- Interaction_log: Stores interaction logs of learners.
- Quest: Stores quest information.
- QuestReward: Stores rewards for quests.
- Skill_Mastery: Stores skill mastery related to quests.
- Collaborative: Stores collaborative quest information.
- LearnerCollaboration: Stores collaboration information for learners.
- LearnerMastery: Stores mastery information for learners.
- Badge: Stores badge information.
- Discussion_forum: Stores discussion forum information.
- Survey: Stores survey information.
- SurveyQuestions: Stores survey questions.
- LearnerDiscussion: Stores discussion posts by learners.
- FilledSurvey: Stores filled survey responses.
- Achievement: Stores achievement information.
- SkillProgression: Stores skill progression information.
- Leaderboard: Stores leaderboard information.
- Ranking: Stores ranking information for learners.
- Learning_goal: Stores learning goals.
- LearnersGoals: Stores information about learners' goals.

2. InsertScript.sql

This script contains the SQL queries to insert initial data into the tables created by TableScript.sql. Below is a summary of the insertion purposes:

Inserts initial learners, skills, learning preferences, personal profiles, health conditions, courses, course prerequisites, modules, target traits, module content, content library, course enrollments, assessments, taken assessments, instructors, teaching assignments, learning paths, path reviews, learning activities, emotional feedback, emotional feedback reviews,

notifications, received notifications, rewards, interaction logs, quests, quest rewards, skill mastery information, collaborative quest information, learner collaboration information, learner mastery information, badges, discussion forums, surveys, survey questions, discussion posts by learners, filled survey responses, achievements, skill progression information, learning goals, learner goals, leaderboards, and rankings.

3. ProcedureScript.sql

This script contains the SQL queries to create stored procedures for various operations. Below is a list of stored procedures and their purposes:

ViewInfo: View learner information.
LearnerInfo: View personal profile of a learner.
EmotionalState: Output emotional state of a learner.
LogDetails: View interaction log details of a learner.
InstructorReview: View emotional feedback reviewed by an instructor.
CourseRemove: Remove a course and its dependent data.
Highestgrade: Get the highest grade in assessments.
InstructorCount: View count of instructors for each course.
ViewNot: View notifications for a learner.
CreateDiscussion: Create a new discussion forum.
RemoveBadge: Remove a badge.
CriteriaDelete: Delete a quest by criteria.
NotificationUpdate: Update notification read status.
EmotionalTrendAnalysis: Analyze emotional trend for a course and module.
SkillLearners: View learners with a specific skill.
NewActivity: Add a new learning activity.
NewAchievement: Add a new achievement.
LearnerBadge: View learners with a specific badge.
NewPath: Create a new learning path.
TakenCourses: View courses taken by a learner.
CollaborativeQuest: Create a new collaborative quest.
DeadlineUpdate: Update deadline for a quest.
GradeUpdate: Update grade for an assessment.
AssessmentNot: Send notification for an assessment.
NewGoal: Create a new learning goal.
LearnersCourses: View learners enrolled in a course taught by an instructor.
LastActive: View last active date of a discussion forum.
CommonEmotionalState: View the most common emotional state.
ModuleDifficulty: View modules sorted by difficulty level.

Profeciencylevel: View proficiency level of a learner in a skill.
ProfeciencyUpdate: Update proficiency level of a learner in a skill.
LeastBadge: View learner with the least number of badges.
PreferedType: View the most common learning preference type.
AssessmentAnalytics: View analytics for assessments in a course and module.

2. Course station

Contains the ASP.NET MVC C# project itself

1. Models

The models in this directory were automatically generated using ASP.NET from the SQL queries in the SRC Queries folder. These models represent the various entities and relationships defined in the database schema, providing a structured way to interact with the underlying data in the application.

For the command for the models' automatic generation, refer to the README in the GitHub repository

2. Views

an overview and detailed documentation for every .cshtml file found in the Course station/Course station/Views directory of the Educational-platform repository.

1. Overview

The .cshtml files in this repository act as ASP.NET MVC views by combining HTML markup with C# code to render dynamic web pages. Their primary responsibilities include:
- Rendering the user interface.
- Integrating with controller actions.
- Binding to data models.
- Maintaining consistency via layout inheritance and shared partial views.

2. Template for .cshtml Files

For each .cshtml file in this directory, consider documenting the following details:

- Filename: The exact name of the file (e.g., _Layout.cshtml, Index.cshtml).
- Purpose: A short description of the role of the file in the application.
- Controller Association: Which controller and action method uses this view (if applicable).
- Model: The type or structure of the model data bound to the view.
- Layout and Sections:
  - The shared layout used (e.g., _Layout.cshtml) or indicates if the file defines its own layout.
  - Any special sections (e.g., header, footer, scripts) that are customized.
- Dependencies:
  - Any partial views or components that are included.
  - Linked CSS, JavaScript, or external resources.
- Notes: Additional information such as performance considerations, potential future changes, or unique implementation details.

Index.cshtml
- Filename: Index.cshtml
- Purpose:
  - Serves as the primary landing or list view for a particular section, such as displaying a list of courses.
- Controller Association:
  - Generally paired with the Index action of its relevant controller.
- Model:
  - Typically utilizes a collection or a view model containing the data to be displayed.
- Layout and Sections:
  - Inherits from _Layout.cshtml for consistency in the user interface.
- Dependencies:
  - May include partial views (e.g., for rendering course summaries) and client-side libraries.
- Notes:
  - Ensure data is properly retrieved and passed to the view to prevent runtime errors.

Details.cshtml
- Filename: Details.cshtml
- Purpose:
  - Displays detailed information for a single entity, such as a course or a specific educational module.
- Controller Association:
  - Tied to the Details action in the relevant controller.
- Model:
  - Utilizes a strongly-typed model representing the detailed entity.

- Layout and Sections:
  - Uses the shared layout and might include additional sections for enhanced functionality.
- Dependencies:
  - Can leverage partial views for related data (e.g., comments or additional resource details).
- Notes:
  - Ensure that all required data fields are checked for null values to avoid runtime exceptions.

Create.cshtml and Edit.cshtml
- Filename: Create.cshtml / Edit.cshtml
- Purpose:
  - Create.cshtml: Provides a form for adding a new entity.
  - Edit.cshtml: Presents a form for modifying an existing entity.
- Controller Association:
  - Typically associated with the Create or Edit actions in the corresponding controller.
- Model:
  - Uses view models or form models capturing the necessary fields for creation or editing.
- Layout and Sections:
  - Inherits layout from _Layout.cshtml, utilizing its structure and shared sections.
- Dependencies:
  - Relies on client-side and server-side validation routines.
  - May reuse partial views for form elements.
- Notes:
  - Maintain robust validation and error handling to ensure data integrity.

Partial Views (e.g., _Navigation.cshtml, _Footer.cshtml)
- Filename: Files prefixed with an underscore (e.g., _Navigation.cshtml, _Footer.cshtml)
- Purpose:
  - Serve as reusable components that can be embedded in multiple views.
- Controller Association:
  - Generally do not directly associate with a controller action.
- Model:
  - May be strongly-typed or use dynamic data depending on the context.
- Layout and Sections:
  - Integrated into broader views using methods like @Html.Partial("_Navigation").
- Dependencies:
  - Usually, rely on data provided by the parent view.

- Notes:
  - Modular design is key for ease of maintenance and updates.

5. Key Views

Views/PersonalProfile/Delete.cshtml

Summary: Displays a confirmation page to delete a personal profile. **Dependencies:**

- Models.PersonalProfile
- asp-action="DeleteConfirmed"

Views/PersonalProfile/Edit.cshtml

Summary: Displays the form to edit an existing personal profile. **Dependencies:**

- Models.PersonalProfile
- asp-action="Edit"

Views/Notifications/Details.cshtml

Summary: Displays details of a specific notification. **Dependencies:**

- Models.Notification
- asp-action="Edit"
- asp-action="Index"

Views/Instructor/CreateCourse.cshtml

Summary: Displays the form to create a new course. **Dependencies:**

- Models.Course
- asp-action="CreateCourse"
- asp-action="ManageCourses"
- _ValidationScriptsPartial

Views/Instructor/CreateAssessment.cshtml

Summary: Displays the form to create a new assessment. **Dependencies:**

- Models.Assessment
- asp-action="CreateAssessment"

- ViewBag.Modules

Views/Home/Credits.cshtml

Summary: Displays the credits page. **Dependencies:**

- ~/css/credits.css
- ~/js/credits.js

Views/EmotionalFeedback/EmotionalTrendAnalysisResult.cshtml

Summary: Displays the results of an emotional trend analysis. **Dependencies:**

- IEnumerable<Controllers.EmotionalFeedbackController.EmotionalTrendAnalysisResult>

Views/Instructor/Details.cshtml

Summary: Displays the profile details of an instructor. **Dependencies:**

- Models.Instructor
- ~/css/custim.css
- https://cdnjs.cloudflare.com/ajax/libs/animate.css/4.1.1/animate.min.css
- https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0/js/all.min.js

Views/Instructor/DeleteQuest.cshtml

Summary: Displays the form to delete a completed quest. **Dependencies:**

- asp-action="DeleteCompletedQuest"

Views/Instructor/LogIn.cshtml

Summary: Displays the login form for instructors. **Dependencies:**

- Models.Instructor
- _ValidationScriptsPartial
- ~/css/login.css

Views/Learners/AvailableQuests.cshtml

Summary: Displays a list of available quests for a learner. **Dependencies:**

- Controllers.LearnersController.AvailableQuestsViewModel
- asp-controller="Learners"
- asp-action="JoinQuest"

Views/Notifications/Create.cshtml

Summary: Displays the form to create a new notification. **Dependencies:**

- Models.Notification
- asp-action="Create"
- asp-action="Index"

Views/Instructor/AssignQuest.cshtml

Summary: Displays the form to assign a quest to a learner. **Dependencies:**

- ViewBag.Quests
- ViewBag.Learners

Views/Instructor/AddActivity.cshtml

Summary: Displays the form to add a new learning activity. **Dependencies:**

- Models.LearningActivity
- ViewBag.Modules

Views/Instructor/AddAchievement.cshtml

Summary: Displays the form to add a new achievement. **Dependencies:**

- Models.Achievement
- ViewBag.Learners
- ViewBag.Badges
- _ValidationScriptsPartial

Views/Home/Privacy.cshtml

Summary: Displays the privacy policy page. **Dependencies:** None

Views/Home/Index.cshtml

Summary: Displays the home page of the application. **Dependencies:**

- ~/js/typing-effect.js
- ~/css/bootstrap.min.css

Views/Home/AdminLogin.cshtml

Summary: Displays the admin login form. **Dependencies:**

- _ValidationScriptsPartial

Views/Instructor/DeleteCourse.cshtml

Summary: Displays the confirmation page to delete a course. **Dependencies:**

- Models.Course

Views/Home/LogIn.cshtml

Summary: Displays the login selection page. **Dependencies:**

- ~/css/bootstrap.min.css

Views/EmotionalFeedback/EmotionalTrendAnalysis.cshtml

Summary: Displays the form to input data for emotional trend analysis. **Dependencies:** None

Views/Learners/CreateFeedback.cshtml

Summary: Displays the form to create new emotional feedback. **Dependencies:**

- Models.EmotionalFeedback

Views/Instructor/SendNotification.cshtml

Summary: Displays the form to send a notification. **Dependencies:**

- ViewBag.Courses

Views/Instructor/DeleteConfirmation.cshtml

Summary: Displays the confirmation page to delete an instructor. **Dependencies:**

- Models.Instructor

Views/Admin/AdminPage.cshtml

Summary: Displays the admin page with links to manage various entities. **Dependencies:** None

3. Contollers

ModuleController

Summary: Manages modules within the application.

- **Index:** Retrieves and displays a list of all modules.
- **Details:** Displays details of a specific module.
- **Create (GET):** Displays the form to create a new module.
- **Create (POST):** Handles the creation of a new module.
- **Delete (GET):** Displays the confirmation page to delete a module.
- **DeleteConfirmed (POST):** Handles the deletion of a module.
- **ModuleExists:** Checks if a module exists by its ID.

NotificationsController

Summary: Manages notifications for users.

- **Index:** Retrieves and displays notifications based on user type.
- **IndexLearner:** Displays notifications for learners.
- **IndexInstructor:** Displays notifications for instructors.
- **IndexAdmin:** Displays notifications for admins.
- **NotUser:** Redirects to the learner creation page.
- **Details:** Displays details of a specific notification.
- **Create (GET):** Displays the form to create a new notification.
- **Create (POST):** Handles the creation of a new notification.
- **Edit (GET):** Displays the form to edit a notification.
- **Edit (POST):** Handles the editing of a notification.
- **Delete (GET):** Displays the confirmation page to delete a notification.
- **DeleteConfirmed (POST):** Handles the deletion of a notification.
- **NotificationExists:** Checks if a notification exists by its ID.
- **MarkAsRead:** Marks a notification as read.

PathreviewController

Summary: Manages path reviews within the application.

- **Index:** Retrieves and displays a list of all path reviews.
- **Details:** Displays details of a specific path review.
- **Create (GET):** Displays the form to create a new path review.
- **Create (POST):** Handles the creation of a new path review.

- **Edit (GET):** Displays the form to edit a path review.
- **Edit (POST):** Handles the editing of a path review.
- **Delete (GET):** Displays the confirmation page to delete a path review.
- **DeleteConfirmed (POST):** Handles the deletion of a path review.
- **PathreviewExists:** Checks if a path review exists by its composite key.

PersonalProfileController

Summary: Manages personal profiles for learners.

- **Index:** Retrieves and displays a list of all personal profiles.
- **Create (GET):** Displays the form to create a new personal profile.
- **Create (POST):** Handles the creation of a new personal profile.
- **Edit (GET):** Displays the form to edit a personal profile.
- **Edit (POST):** Handles the editing of a personal profile.
- **Delete (GET):** Displays the confirmation page to delete a personal profile.
- **DeleteConfirmed (POST):** Handles the deletion of a personal profile.
- **PersonalProfileExists:** Checks if a personal profile exists by its ID.

QuestRewardController

Summary: Manages quest rewards within the application.

- **Index:** Retrieves and displays a list of all quest rewards.
- **Details:** Displays details of a specific quest reward.
- **Create (GET):** Displays the form to create a new quest reward.
- **Create (POST):** Handles the creation of a new quest reward.
- **Edit (GET):** Displays the form to edit a quest reward.
- **Edit (POST):** Handles the editing of a quest reward.
- **Delete (GET):** Displays the confirmation page to delete a quest reward.
- **DeleteConfirmed (POST):** Handles the deletion of a quest reward.
- **QuestRewardExists:** Checks if a quest reward exists by its composite key.

QuestsController

Summary: Manages quests within the application.

- **Index:** Retrieves and displays a list of all quests.
- **Details:** Displays details of a specific quest.
- **Create (GET):** Displays the form to create a new quest.
- **Create (POST):** Handles the creation of a new quest.

- **Edit (GET):** Displays the form to edit a quest.
- **Edit (POST):** Handles the editing of a quest.
- **Delete (GET):** Displays the confirmation page to delete a quest.
- **DeleteConfirmed (POST):** Handles the deletion of a quest.
- **QuestExists:** Checks if a quest exists by its ID.
- **Join:** Handles the joining of a quest by a learner.
- **Participants:** Displays the participants of a quest.

RankingController

Summary: Manages rankings within the application.

- **Index:** Retrieves and displays a list of all rankings.
- **Details:** Displays details of a specific ranking.
- **Create (GET):** Displays the form to create a new ranking.
- **Create (POST):** Handles the creation of a new ranking.
- **Edit (GET):** Displays the form to edit a ranking.
- **Edit (POST):** Handles the editing of a ranking.
- **Delete (GET):** Displays the confirmation page to delete a ranking.
- **DeleteConfirmed (POST):** Handles the deletion of a ranking.
- **RankingExists:** Checks if a ranking exists by its composite key.

RewardController

Summary: Manages rewards within the application.

- **Index:** Retrieves and displays a list of all rewards.
- **Details:** Displays details of a specific reward.
- **Create (GET):** Displays the form to create a new reward.
- **Create (POST):** Handles the creation of a new reward.
- **Edit (GET):** Displays the form to edit a reward.
- **Edit (POST):** Handles the editing of a reward.
- **Delete (GET):** Displays the confirmation page to delete a reward.
- **DeleteConfirmed (POST):** Handles the deletion of a reward.
- **RewardExists:** Checks if a reward exists by its ID.

SkillController

Summary: Manages skills within the application.

- **Index:** Retrieves and displays a list of all skills.

- **Details:** Displays details of a specific skill.
- **Create (GET):** Displays the form to create a new skill.
- **Create (POST):** Handles the creation of a new skill.
- **Edit (GET):** Displays the form to edit a skill.
- **Edit (POST):** Handles the editing of a skill.
- **Delete (GET):** Displays the confirmation page to delete a skill.
- **DeleteConfirmed (POST):** Handles the deletion of a skill.
- **SkillExists:** Checks if a skill exists by its composite key.

SkillMasteryController

Summary: Manages skill masteries within the application.

- **Index:** Retrieves and displays a list of all skill masteries.
- **Details:** Displays details of a specific skill mastery.
- **Create (GET):** Displays the form to create a new skill mastery.
- **Create (POST):** Handles the creation of a new skill mastery.
- **Edit (GET):** Displays the form to edit a skill mastery.
- **Edit (POST):** Handles the editing of a skill mastery.
- **Delete (GET):** Displays the confirmation page to delete a skill mastery.
- **DeleteConfirmed (POST):** Handles the deletion of a skill mastery.
- **SkillMasteryExists:** Checks if a skill mastery exists by its composite key.

SkillProgressionController

Summary: Manages skill progressions within the application.

- **Index:** Retrieves and displays a list of all skill progressions.
- **Details:** Displays details of a specific skill progression.
- **Create (GET):** Displays the form to create a new skill progression.
- **Create (POST):** Handles the creation of a new skill progression.
- **Edit (GET):** Displays the form to edit a skill progression.
- **Edit (POST):** Handles the editing of a skill progression.
- **Delete (GET):** Displays the confirmation page to delete a skill progression.
- **DeleteConfirmed (POST):** Handles the deletion of a skill progression.
- **SkillProgressionExists:** Checks if a skill progression exists by its ID.

SurveyController

Summary: Manages surveys within the application.

- **Index:** Retrieves and displays a list of all surveys.
- **Details:** Displays details of a specific survey.
- **Create (GET):** Displays the form to create a new survey.
- **Create (POST):** Handles the creation of a new survey.
- **Edit (GET):** Displays the form to edit a survey.
- **Edit (POST):** Handles the editing of a survey.
- **Delete (GET):** Displays the confirmation page to delete a survey.
- **DeleteConfirmed (POST):** Handles the deletion of a survey.
- **SurveyExists:** Checks if a survey exists by its ID.

SurveyQuestionController

Summary: Manages survey questions within the application.

- **Index:** Retrieves and displays a list of all survey questions.
- **Details:** Displays details of a specific survey question.
- **Create (GET):** Displays the form to create a new survey question.
- **Create (POST):** Handles the creation of a new survey question.
- **Edit (GET):** Displays the form to edit a survey question.
- **Edit (POST):** Handles the editing of a survey question.
- **Delete (GET):** Displays the confirmation page to delete a survey question.
- **DeleteConfirmed (POST):** Handles the deletion of a survey question.
- **SurveyQuestionExists:** Checks if a survey question exists by its composite key.

TakenAssessmentController

Summary: Manages taken assessments within the application.

- **Index:** Retrieves and displays a list of all taken assessments.
- **Details:** Displays details of a specific taken assessment.
- **Create (GET):** Displays the form to create a new taken assessment.
- **Create (POST):** Handles the creation of a new taken assessment.
- **Edit (GET):** Displays the form to edit a taken assessment.
- **Edit (POST):** Handles the editing of a taken assessment.
- **Delete (GET):** Displays the confirmation page to delete a taken assessment.
- **DeleteConfirmed (POST):** Handles the deletion of a taken assessment.
- **TakenAssessmentExists:** Checks if a taken assessment exists by its composite key.

TargetTraitController

Summary: Manages target traits within the application.

- **Index:** Retrieves and displays a list of all target traits.
- **Details:** Displays details of a specific target trait.
- **Create (GET):** Displays the form to create a new target trait.
- **Create (POST):** Handles the creation of a new target trait.
- **Edit (GET):** Displays the form to edit a target trait.
- **Edit (POST):** Handles the editing of a target trait.
- **Delete (GET):** Displays the confirmation page to delete a target trait.
- **DeleteConfirmed (POST):** Handles the deletion of a target trait.
- **TargetTraitExists:** Checks if a target trait exists by its composite key.

LeaderboardController

Summary: Manages leaderboard-related actions, including viewing, creating, editing, and deleting leaderboards.

Methods:

- Index(): Displays a list of all leaderboards.
- Details(int? id): Shows details of a specific leaderboard, including its rankings.
- Create(): Displays a form to create a new leaderboard.
- Create([Bind(...)]): Handles the creation of a new leaderboard.
- Edit(int? id): Displays a form to edit an existing leaderboard.
- Edit(int id, [Bind(...)]): Handles the editing of an existing leaderboard.
- Delete(int? id): Displays a confirmation page to delete a leaderboard.
- DeleteConfirmed(int id): Handles the deletion of a leaderboard.
- LeaderboardExists(int id): Checks if a leaderboard exists.

LearnerCollaborationController

Summary: Manages learner collaborations on quests, including viewing, creating, editing, and deleting collaborations.

Methods:

- Index(): Displays a list of all learner collaborations.
- Details(int? learnerId, int? questId): Shows details of a specific learner collaboration.
- Create(): Displays a form to create a new learner collaboration.
- Create([Bind(...)]): Handles the creation of a new learner collaboration.
- Edit(int? learnerId, int? questId): Displays a form to edit an existing learner collaboration.
- Edit(int learnerId, int questId, [Bind(...)]): Handles the editing of an existing learner collaboration.

- Delete(int? learnerId, int? questId): Displays a confirmation page to delete a learner collaboration.
- DeleteConfirmed(int learnerId, int questId): Handles the deletion of a learner collaboration.
- LearnerCollaborationExists(int learnerId, int questId): Checks if a learner collaboration exists.

LearnerDiscussionController

Summary: Manages learner discussions in forums, including viewing, creating, editing, and deleting discussions.

Methods:

- Index(): Displays a list of all learner discussions.
- Details(int? forumId, int? learnerId, string post): Shows details of a specific learner discussion.
- Create(): Displays a form to create a new learner discussion.
- Create([Bind(...)]): Handles the creation of a new learner discussion.
- Edit(int? forumId, int? learnerId, string post): Displays a form to edit an existing learner discussion.
- Edit(int forumId, int learnerId, string post, [Bind(...)]): Handles the editing of an existing learner discussion.
- Delete(int? forumId, int? learnerId, string post): Displays a confirmation page to delete a learner discussion.
- DeleteConfirmed(int forumId, int learnerId, string post): Handles the deletion of a learner discussion.
- LearnerDiscussionExists(int forumId, int learnerId, string post): Checks if a learner discussion exists.

LearnerMasteryController

Summary: Manages learner mastery of quests, including viewing, creating, editing, and deleting mastery records.

Methods:

- Index(): Displays a list of all learner masteries.
- Details(int? learnerId, int? questId): Shows details of a specific learner mastery.
- Create(): Displays a form to create a new learner mastery.
- Create([Bind(...)]): Handles the creation of a new learner mastery.
- Edit(int? learnerId, int? questId): Displays a form to edit an existing learner mastery.
- Edit(int learnerId, int questId, [Bind(...)]): Handles the editing of an existing learner mastery.
- Delete(int? learnerId, int? questId): Displays a confirmation page to delete a learner mastery.
- DeleteConfirmed(int learnerId, int questId): Handles the deletion of a learner mastery.

- LearnerMasteryExists(int learnerId, int questId): Checks if a learner mastery exists.

LearnersController

Summary: Manages learner-related actions, including viewing, creating, editing, and deleting learners, as well as handling learner-specific functionalities like login and course enrollment.

Methods:

- Home(): Displays the home page for a learner.
- Index(): Displays a list of all learners (admin only).
- Details(int? id): Shows details of a specific learner.
- UpdateQuestProgress([FromBody] UpdateQuestProgressModel model): Updates the progress of a quest for a learner.
- AvailableQuests(int learnerId): Displays available quests for a learner.
- JoinQuest(int learnerId, int questId): Displays a form to join a quest.
- ConfirmJoinQuest(int learnerId, int questId): Handles the joining of a quest.
- Create(): Displays a form to create a new learner.
- Create([Bind(...)]): Handles the creation of a new learner.
- Edit(int? id): Displays a form to edit an existing learner.
- Edit(int id, [Bind(...)]): Handles the editing of an existing learner.
- Delete(int? id): Displays a confirmation page to delete a learner.
- DeleteConfirmed(int id): Handles the deletion of a learner.
- Login(): Displays the login page for a learner.
- Login(Learner model): Handles the login process for a learner.
- LearnerExists(int id): Checks if a learner exists.
- ViewInfo(int learnerId): Retrieves learner information.
- MarkAsRead(int learnerId, int id): Marks a notification as read.
- UpdateProfile(): Displays a form to update the learner's profile.
- UploadProfilePicture(int learnerId, IFormFile profilePicture): Handles the uploading of a profile picture.
- CheckPrerequisites(int learnerId, int courseId): Checks if a learner meets the prerequisites for a course.
- ModuleTraits(string targetTrait, int courseId): Displays modules that train specific traits.
- LeaderboardRank(int leaderboardId): Displays participants in a leaderboard and their ranking.
- SubmitEmotionalFeedback(int activityId, int learnerId, string emotionalState): Submits emotional feedback for an activity.
- SkillsProficiency(int learnerId): Displays skill proficiency levels for a learner.
- ViewScore(int learnerId, int assessmentId): Displays the assessment score for a learner.
- AssessmentsList(int learnerId, int courseId): Displays a list of assessments for a course.
- RegisterCourse(int learnerId, int courseId): Registers a learner for a course.

- Enroll(int learnerId): Displays a form to enroll in a course.
- Enroll(EnrollViewModel model): Handles the enrollment process for a course.
- ViewPrerequisites(int learnerId, int courseId): Displays prerequisites for a course.
- EnrolledCourses(int learnerId): Displays courses a learner is enrolled in.
- QuestProgress(): Displays the progress of quests.
- JoinCollaborativeQuest(int learnerId, int questId): Handles the joining of a collaborative quest.
- AchievementProgress(int learnerId): Displays the achievement progress for a learner.
- ActiveQuestParticipants(int learnerId): Displays active quest participants.

LearningActivityController

Summary: Manages learning activities, including viewing, creating, editing, and deleting activities.

Methods:

- Index(): Displays a list of all learning activities.
- Details(int? id): Shows details of a specific learning activity.
- Create(): Displays a form to create a new learning activity.
- Create([Bind(...)]): Handles the creation of a new learning activity.
- Edit(int? id): Displays a form to edit an existing learning activity.
- Edit(int id, [Bind(...)]): Handles the editing of an existing learning activity.
- Delete(int? id): Displays a confirmation page to delete a learning activity.
- DeleteConfirmed(int id): Handles the deletion of a learning activity.
- LearningActivityExists(int id): Checks if a learning activity exists.

LearningGoalController

Summary: Manages learning goals, including viewing, creating, editing, and deleting goals.

Methods:

- Index(): Displays a list of all learning goals.
- Details(int? id): Shows details of a specific learning goal.
- Create(): Displays a form to create a new learning goal.
- Create([Bind(...)]): Handles the creation of a new learning goal.
- Edit(int? id): Displays a form to edit an existing learning goal.
- Edit(int id, [Bind(...)]): Handles the editing of an existing learning goal.
- Delete(int? id): Displays a confirmation page to delete a learning goal.
- DeleteConfirmed(int id): Handles the deletion of a learning goal.
- LearningGoalExists(int id): Checks if a learning goal exists.

LearningPathController

Summary: Manages learning paths, including viewing, creating, editing, and deleting paths.

Methods:

- Index(): Displays a list of all learning paths.
- Details(int? id): Shows details of a specific learning path.
- Create(): Displays a form to create a new learning path.
- Create([Bind(...)]): Handles the creation of a new learning path.
- Edit(int? id): Displays a form to edit an existing learning path.
- Edit(int id, [Bind(...)]): Handles the editing of an existing learning path.
- Delete(int? id): Displays a confirmation page to delete a learning path.
- DeleteConfirmed(int id): Handles the deletion of a learning path.
- LearningPathExists(int id): Checks if a learning path exists.

LearningPrefrenceController

Summary: Manages learning preferences, including viewing, creating, editing, and deleting preferences.

Methods:

- Index(): Displays a list of all learning preferences.
- Details(int? learnerId, string prefrences): Shows details of a specific learning preference.
- Create(): Displays a form to create a new learning preference.
- Create([Bind(...)]): Handles the creation of a new learning preference.
- Edit(int? learnerId, string prefrences): Displays a form to edit an existing learning preference.
- Edit(int learnerId, string prefrences, [Bind(...)]): Handles the editing of an existing learning preference.
- Delete(int? learnerId, string prefrences): Displays a confirmation page to delete a learning preference.
- DeleteConfirmed(int learnerId, string prefrences): Handles the deletion of a learning preference.
- LearningPrefrenceExists(int learnerId, string prefrences): Checks if a learning preference exists.

ModuleContentController

Summary: Manages module content, including viewing, creating, editing, and deleting content.

Methods:

- Index(): Displays a list of all module content.

- Details(int? moduleId, int? courseId, string contetntType): Shows details of a specific module content.
- Create(): Displays a form to create new module content.
- Create([Bind(...)]): Handles the creation of new module content.
- Edit(int? moduleId, int? courseId, string contetntType): Displays a form to edit existing module content.
- Edit(int moduleId, int courseId, string contetntType, [Bind(...)]): Handles the editing of existing module content.
- Delete(int? moduleId, int? courseId, string contetntType): Displays a confirmation page to delete module content.
- DeleteConfirmed(int moduleId, int courseId, string contetntType): Handles the deletion of module content.
- ModuleContentExists(int moduleId, int courseId, string contetntType): Checks if module content exists.

CourseEnrollmentController

Summary: Manages course enrollments, including listing, viewing details, creating, editing, and deleting enrollments.

- **Index:** Lists all course enrollments.
- **Details:** Displays details of a specific course enrollment.
- **Create (GET):** Displays the form to create a new course enrollment.
- **Create (POST):** Handles the creation of a new course enrollment.
- **Edit (GET):** Displays the form to edit an existing course enrollment.
- **Edit (POST):** Handles the update of an existing course enrollment.
- **Delete (GET):** Displays the confirmation page to delete a course enrollment.
- **Delete (POST):** Handles the deletion of a course enrollment.
- **CourseEnrollmentExists:** Checks if a course enrollment exists by ID.

CoursePrerequisiteController

Summary: Manages course prerequisites, including listing, viewing details, creating, editing, and deleting prerequisites.

- **Index:** Lists all course prerequisites.
- **Details:** Displays details of a specific course prerequisite.
- **Create (GET):** Displays the form to create a new course prerequisite.
- **Create (POST):** Handles the creation of a new course prerequisite.
- **Edit (GET):** Displays the form to edit an existing course prerequisite.
- **Edit (POST):** Handles the update of an existing course prerequisite.

- **Delete (GET):** Displays the confirmation page to delete a course prerequisite.
- **Delete (POST):** Handles the deletion of a course prerequisite.
- **CoursePrerequisiteExists:** Checks if a course prerequisite exists by course ID and prerequisite.
- **CheckPrerequisitesCompleted:** Checks if a learner has completed the prerequisites for a course.

DiscussionForumController

Summary: Manages discussion forums, including listing, viewing details, creating, editing, and deleting forums.

- **Index:** Lists all discussion forums.
- **Details:** Displays details of a specific discussion forum.
- **Create (GET):** Displays the form to create a new discussion forum.
- **Create (POST):** Handles the creation of a new discussion forum.
- **Edit (GET):** Displays the form to edit an existing discussion forum.
- **Edit (POST):** Handles the update of an existing discussion forum.
- **Delete (GET):** Displays the confirmation page to delete a discussion forum.
- **Delete (POST):** Handles the deletion of a discussion forum.
- **DiscussionForumExists:** Checks if a discussion forum exists by ID.

EmotionalFeedbackController

Summary: Manages emotional feedback, including listing, viewing details, creating, editing, and deleting feedback.

- **Index:** Lists all emotional feedback.
- **Details:** Displays details of a specific emotional feedback.
- **Create (GET):** Displays the form to create a new emotional feedback.
- **Create (POST):** Handles the creation of a new emotional feedback.
- **Edit (GET):** Displays the form to edit an existing emotional feedback.
- **Edit (POST):** Handles the update of an existing emotional feedback.
- **Delete (GET):** Displays the confirmation page to delete an emotional feedback.
- **Delete (POST):** Handles the deletion of an emotional feedback.
- **EmotionalFeedbackExists:** Checks if an emotional feedback exists by ID.
- **Trend:** Redirects to the emotional feedback review index.
- **EmotionalTrendAnalysis (GET):** Displays the form for emotional trend analysis.
- **EmotionalTrendAnalysis (POST):** Handles the analysis of emotional trends.

EmotionalfeedbackReviewController

Summary: Manages reviews of emotional feedback, including listing, viewing details, creating, editing, and deleting reviews.

- **Index:** Lists all emotional feedback reviews.
- **Details:** Displays details of a specific emotional feedback review.
- **Create (GET):** Displays the form to create a new emotional feedback review.
- **Create (POST):** Handles the creation of a new emotional feedback review.
- **Edit (GET):** Displays the form to edit an existing emotional feedback review.
- **Edit (POST):** Handles the update of an existing emotional feedback review.
- **Delete (GET):** Displays the confirmation page to delete an emotional feedback review.
- **Delete (POST):** Handles the deletion of an emotional feedback review.
- **EmotionalfeedbackReviewExists:** Checks if an emotional feedback review exists by feedback ID and instructor ID.
- **EmotionalTrendAnalysis:** Handles the analysis of emotional trends for a course and module.

FilledSurveyController

Summary: Manages filled surveys, including listing, viewing details, creating, editing, and deleting surveys.

- **Index:** Lists all filled surveys.
- **Details:** Displays details of a specific filled survey.
- **Create (GET):** Displays the form to create a new filled survey.
- **Create (POST):** Handles the creation of a new filled survey.
- **Edit (GET):** Displays the form to edit an existing filled survey.
- **Edit (POST):** Handles the update of an existing filled survey.
- **Delete (GET):** Displays the confirmation page to delete a filled survey.
- **Delete (POST):** Handles the deletion of a filled survey.
- **FilledSurveyExists:** Checks if a filled survey exists by survey ID, question, and learner ID.

HealthConditionController

Summary: Manages health conditions, including listing, viewing details, creating, editing, and deleting conditions.

- **Index:** Lists all health conditions.
- **Details:** Displays details of a specific health condition.
- **Create (GET):** Displays the form to create a new health condition.
- **Create (POST):** Handles the creation of a new health condition.

- **Edit (GET):** Displays the form to edit an existing health condition.
- **Edit (POST):** Handles the update of an existing health condition.
- **Delete (GET):** Displays the confirmation page to delete a health condition.
- **Delete (POST):** Handles the deletion of a health condition.
- **HealthConditionExists:** Checks if a health condition exists by learner ID, profile ID, and condition.

HomeController

Summary: Manages the home page, including displaying the index, privacy, and error pages, and handling admin login.

- **Index:** Displays the home page.
- **Privacy:** Displays the privacy policy page.
- **Error:** Displays the error page.
- **AdminLogin (GET):** Displays the admin login form.
- **AdminLogin (POST):** Handles admin login.
- **Credits:** Displays the credits page.
- **Delete:** Handles the deletion of an item.
- **Details:** Displays details of an item.
- **Create:** Displays the form to create an item.
- **LogIn:** Displays the login form.
- **SubmitForm:** Handles form submission.
- **ThankYou:** Displays the thank you page.

InteractionLogController

Summary: Manages interaction logs, including listing, viewing details, creating, editing, and deleting logs.

- **Index:** Lists all interaction logs.
- **Details:** Displays details of a specific interaction log.
- **Create (GET):** Displays the form to create a new interaction log.
- **Create (POST):** Handles the creation of a new interaction log.
- **Edit (GET):** Displays the form to edit an existing interaction log.
- **Edit (POST):** Handles the update of an existing interaction log.
- **Delete (GET):** Displays the confirmation page to delete an interaction log.
- **Delete (POST):** Handles the deletion of an interaction log.
- **InteractionLogExists:** Checks if an interaction log exists by ID.

InstructorController

Summary: Manages instructors, including listing, viewing details, creating, editing, and deleting instructors, and handling various instructor-related actions.

- **Home:** Displays the instructor's home page.
- **Index:** Lists all instructors.
- **Details:** Displays details of a specific instructor.
- **Create (GET):** Displays the form to create a new instructor.
- **Create (POST):** Handles the creation of a new instructor.
- **Edit (GET):** Displays the form to edit an existing instructor.
- **Edit (POST):** Handles the update of an existing instructor.
- **Delete (GET):** Displays the confirmation page to delete an instructor.
- **Delete (POST):** Handles the deletion of an instructor.
- **Login (GET):** Displays the instructor login form.
- **Login (POST):** Handles instructor login.
- **Logout:** Handles instructor logout.
- **ManageCourses:** Lists courses managed by the instructor.
- **CourseDetails:** Displays details of a specific course.
- **EditCourse (GET):** Displays the form to edit an existing course.
- **EditCourse (POST):** Handles the update of an existing course.
- **DeleteCourse (GET):** Displays the confirmation page to delete a course.
- **SendNotification (GET):** Displays the form to send a notification.
- **SendNotification (POST):** Handles sending a notification.
- **GetLearnersByCourse:** Lists learners enrolled in a specific course.
- **UploadProfilePicture:** Handles uploading a profile picture.
- **SkillLearners:** Lists learners with a specific skill.
- **NewActivity:** Adds a new learning activity for a course module.
- **NewAchievement:** Awards a new achievement to a learner.
- **LearnerBadge:** Lists learners who earned a specific badge.
- **NewPath:** Adds a new learning path for a learner.
- **TakenCourses:** Lists courses taken by a learner.
- **CreateCourse (GET):** Displays the form to create a new course.
- **CreateCourse (POST):** Handles the creation of a new course.
- **CreateAssessment (GET):** Displays the form to create a new assessment.
- **CreateAssessment (POST):** Handles the creation of a new assessment.
- **CollaborativeQuest:** Adds a new collaborative quest.
- **DeadlineUpdate:** Updates the deadline of a quest.
- **GradeUpdate:** Updates an assessment grade for a learner.
- **AssessmentNot:** Sends a notification about an upcoming assessment deadline.

- **NewGoal:** Defines a new learning goal for learners.
- **LastActive:** Displays the last active time of a discussion forum.
- **CommonEmotionalState:** Displays the most common emotional state of learners.
- **ModuleDifficulty:** Lists modules of a course sorted by difficulty.
- **ProficiencyLevel:** Displays the skill with the highest proficiency level for a learner.
- **ProficiencyUpdate:** Updates a learner's proficiency level for a skill.
- **LeastBadge:** Displays the learner with the least number of badges earned.
- **PreferredType:** Displays the most preferred learning type of learners.
- **AssessmentAnalytics:** Displays analytics on assessment scores.
- **EmotionalTrendAnalysisIns:** Displays trends in learners' emotional feedback.
- **AddActivity (GET):** Displays the form to add a new activity.
- **AddActivity (POST):** Handles adding a new activity.

LearnerCollaborationController

Summary: Manages learner collaborations, including listing, viewing details, creating, editing, and deleting collaborations.

- **Index:** Lists all learner collaborations.
- **Details:** Displays details of a specific learner collaboration.
- **Create (GET):** Displays the form to create a new learner collaboration.
- **Create (POST):** Handles the creation of a new learner collaboration.
- **Edit (GET):** Displays the form to edit an existing learner collaboration.
- **Edit (POST):** Handles the update of an existing learner collaboration.
- **Delete (GET):** Displays the confirmation page to delete a learner collaboration.
- **Delete (POST):** Handles the deletion of a learner collaboration.
- **LearnerCollaborationExists:** Checks if a learner collaboration exists by learner ID and quest ID.

AccountController

Summary: Manages user account-related actions, including login and role assignment.

- **Login (POST):** Handles user login.
- **AssignRole (POST):** Assigns a role to a user.

AchievementsController

Summary: Manages achievements, including listing, viewing details, creating, editing, and deleting achievements.

- **Index:** Lists all achievements.
- **Details:** Displays details of a specific achievement.
- **Create (GET):** Displays the form to create a new achievement.
- **Create (POST):** Handles the creation of a new achievement.
- **Edit (GET):** Displays the form to edit an existing achievement.
- **Edit (POST):** Handles the update of an existing achievement.
- **Delete (GET):** Displays the confirmation page to delete an achievement.
- **Delete (POST):** Handles the deletion of an achievement.
- **AchievementExists:** Checks if an achievement exists by ID.

AdminController

Summary: Manages admin-related actions, including login, logout, and viewing the admin page.

- **AdminLogin (GET):** Displays the admin login form.
- **Login (POST):** Handles admin login.
- **AdminPage:** Displays the admin page.
- **Logout:** Handles admin logout.
- **Index:** Displays the index page.

AssessmentsController

Summary: Manages assessments, including listing, viewing details, creating, editing, and deleting assessments.

- **Index:** Lists all assessments.
- **Details:** Displays details of a specific assessment.
- **Take (GET):** Displays the form to take an assessment.
- **Take (POST):** Handles the submission of an assessment.
- **ViewScore:** Displays the score of a taken assessment.

BadgeController

Summary: Manages badges, including listing, viewing details, creating, editing, and deleting badges.

- **Index:** Lists all badges.
- **Details:** Displays details of a specific badge.
- **Create (GET):** Displays the form to create a new badge.
- **Create (POST):** Handles the creation of a new badge.
- **Edit (GET):** Displays the form to edit an existing badge.
- **Edit (POST):** Handles the update of an existing badge.

- **Delete (GET):** Displays the confirmation page to delete a badge.
- **Delete (POST):** Handles the deletion of a badge.
- **BadgeExists:** Checks if a badge exists by ID.

ChatbotController

Summary: Manages chatbot interactions, including getting suggestions and chatting.

- **GetSuggestions (POST):** Retrieves suggestions based on user input.
- **Chat (POST):** Handles chatbot responses based on user input.

CollaborativeController

Summary: Manages collaborative quests, including listing, viewing details, creating, editing, and deleting collaborative quests.

- **Index:** Lists all collaborative quests.
- **Details:** Displays details of a specific collaborative quest.
- **Create (GET):** Displays the form to create a new collaborative quest.
- **Create (POST):** Handles the creation of a new collaborative quest.
- **Edit (GET):** Displays the form to edit an existing collaborative quest.
- **Edit (POST):** Handles the update of an existing collaborative quest.
- **Delete (GET):** Displays the confirmation page to delete a collaborative quest.
- **Delete (POST):** Handles the deletion of a collaborative quest.
- **CollaborativeExists:** Checks if a collaborative quest exists by ID.

CourseController

Summary: Manages courses, including listing, viewing details, creating, editing, and deleting courses.

- **Index:** Lists all courses.
- **Details:** Displays details of a specific course.
- **DetailsForLearner:** Displays course details for a learner.
- **Create (GET):** Displays the form to create a new course.
- **Create (POST):** Handles the creation of a new course.
- **Edit (GET):** Displays the form to edit an existing course.
- **Edit (POST):** Handles the update of an existing course.
- **Delete (GET):** Displays the confirmation page to delete a course.
- **Delete (POST):** Handles the deletion of a course.
- **CourseExists:** Checks if a course exists by ID.
- **Modules:** Lists modules of a course.

- **CreateModule (GET):** Displays the form to create a new module.
- **CreateModule (POST):** Handles the creation of a new module.
- **CreateAssessment (GET):** Displays the form to create a new assessment.
- **CreateAssessment (POST):** Handles the creation of a new assessment.
- **TakenCourseDetails:** Displays details of a taken course.

ContentLibraryController

Summary: Manages content libraries, including listing, viewing details, creating, editing, and deleting content libraries.

- **Index:** Lists all content libraries.
- **Details:** Displays details of a specific content library.
- **Create (GET):** Displays the form to create a new content library.
- **Create (POST):** Handles the creation of a new content library.
- **Edit (GET):** Displays the form to edit an existing content library.
- **Edit (POST):** Handles the update of an existing content library.
- **Delete (GET):** Displays the confirmation page to delete a content library.
- **Delete (POST):** Handles the deletion of a content library.
- **ContentLibraryExists:** Checks if a content library exists by ID.

Important Note :

If are required to modify the tables or schema Consider the bindings in the tables models controllers and entity and dependencies